

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине Построение и анализ алгоритмов
ТЕМА: «РЕДАКЦИОННОЕ РАССТОЯНИЕ»

Студент гр. 4343

Миридаштаки Ф.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы

Изучить работу алгоритма Вагнера-Фишера для построения матрицы расстояния Левенштейна и нахождения редакционного предписания.

Задание 1

Над строкой ϵ (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. $replace(\epsilon, a, b)$ – заменить символ a на символ b .
2. $insert(\epsilon, a)$ – вставить в строку символ a (на любую позицию).
3. $delete(\epsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B .

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A ; третья строка – B .

Выходные данные: одно число – минимальная стоимость операций.

Sample Input:

1 1 1
entrance
reenterable

Sample Output:

5

Задание 2

Над строкой ϵ (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. $replace(\epsilon, a, b)$ – заменить символ a на символ b .

2. $insert(\epsilon, a)$ – вставить в строку символ a (на любую позицию).

3. $delete(\epsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки A в строку B .

Пример (все операции стоят одинаково)

М	М	М	Р	І	М	Р	Р
С	О	Н	Н		Е	С	Т
С	О	Н	Е	Н	Е	А	Д

Пример (цена замены 3, остальные операции по 1)

М	М	М	Д	М	І	І	І	І	Д	Д
С	О	Н	Н	Е					С	Т
С	О	Н		Е	Н	Е	А	Д		

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A ; третья строка – B .

Выходные данные: первая строка – последовательность операций (M – совпадение, ничего делать не надо; R – заменить символ на другой; I – вставить символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

Sample Input:

1 1 1

entrance

reenterable

Sample Output:

IMIMMIMMRRM

entrance

reenterable

Задание 3

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк pedestal и stien расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: pedestal -> stal.
- Затем необходимо заменить два последних символа: stal -> stie.
- Потом нужно добавить символ в конец строки: stie -> stien.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($S, 1 \leq |S| \leq 25501 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($T, 1 \leq |T| \leq 25501 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Sample Input:

pedestal
stien

Sample Output:

7

Вариант 2.

«Особый заменитель и особо удаляемый символ»: цена замены на определённый символ отличается от обычной цены замены; цена удаления другого (или того же) определённого символа отличается от обычной цены удаления. Особый заменитель и цена замены на него, особо удаляемый символ и цена его удаления — дополнительные входные данные.

Выполнение работы

Описание алгоритма

Расстояние Левенштейна показывает, в какое количество действий с символами одно слово можно преобразовать в другое, а алгоритм Вагнера-Фишера – это алгоритм нахождения этого расстояния путём составления матрицы расстояний.

Сначала идёт заполнение матрицы расстояний. Она строится на основе двух рассматриваемых слов. Каждое значение в ней – расстояние Левенштейна для двух подстрок, полученных путём обрезания оригинальных строк по индексам строки и столбца. Мы преобразуем первое поданное слово во второе. Рассмотрим основные случаи при заполнении:

$d[0][0] = 0$ – расстояние между двумя пустыми строками – нулевое.

$d[0][j] = j \times \text{insertion_cost}$ (первая строка) – чтобы получить из пустой строки вторую (или её подстроку), нужно выполнить j вставок.

$d[i][0] = \text{сумма стоимостей удаления символов } s1[0..i-1]$ (первый столбец) – чтобы получить из начальной строки (или её подстроки) пустую, нужно выполнить i удалений. В варианте 2 при заполнении первого столбца учитывается особая цена удаления, если удаляемый символ является особо удаляемым.

$d[i][j]$, $i > 0$, $j > 0$ – для остальных ячеек матрицы берётся минимум из определённых вычисленных значений + цена соответствующей операции. Среди рассматриваемых операций:

- значение слева ($d[i][j-1]$) + цена вставки текущего символа,
- значение сверху ($d[i-1][j]$) + цена удаления текущего символа (особая, если символ является особо удаляемым),
- значение слева-сверху ($d[i-1][j-1]$) + цена замены (особая, если целевой символ является особым заменителем); если символы совпадают, то цена замены = 0.

Таким образом, значение в правом нижнем углу матрицы – расстояние Левенштейна для рассматриваемых строк.

Затем находится редакционное предписание – последовательность операций, которые преобразуют первую строку во вторую. Для этого необходимо из конечного значения матрицы (справа снизу) вернуться в начальную (слева сверху) по наиболее оптимальному пути. Берётся минимум из значения + цены операции для левой, верхней и левой верхней ячеек, которые соответственно обозначают операции вставки, удаления и замены (в случае совпадения символов операция не требуется).

Оценка сложности

Временная сложность алгоритма – $O(n \times m)$, где n – длина первой строки, а m – длина второй. Сложность квадратичная, поскольку программа составляет матрицу расстояний размером $(n+1) \times (m+1)$.

Пространственная сложность алгоритма – $O(n \times m)$, поскольку необходимо хранить матрицу расстояний размером $(n+1) \times (m+1)$.

Код программы содержит реализацию следующих функций:

- `decide_costs(src_char, tgt_char, replace_cost, delete_cost, r_char, r_price, d_char, d_price)` – определяет, являются ли обрабатываемые символы особыми, и возвращает актуальные цены замены и удаления для данной пары символов.
- `get_distance_matrix(s1, s2, replace_cost, insert_cost, delete_cost, r_char, r_price, d_char, d_price)` – составляет матрицу расстояний, основываясь на полученных в предыдущих шагах расстояниях и ценах операций, с учётом особых символов варианта 2.
- `traceback_operations(d, s1, s2, replace_cost, insert_cost, delete_cost, r_char, r_price, d_char, d_price)` – обходит матрицу расстояний с правого нижнего угла к левому верхнему, получая последовательность операций, которые ведут к наименьшему расстоянию Левенштейна.

- `check_solution(s1, s2, solution)` – применяет операции, полученные обратным обходом матрицы, к первой строке, так, чтобы в конце она совпала со второй строкой.
- `print_matrix(d, s1, s2)` – выводит матрицу расстояний в читаемом виде.
- `main()` – обрабатывает ввод данных и вызывает остальные функции.

Тестирование

Программа была протестирована на различных входных данных.

Составлена соответствующая таблица:

Таблица 1.

Входные данные	Выходные данные	Комментарий
1 1 1 abc abcdefg	MMMIII 4	Добавление символов
1 1 1 abcdefg abc	MMMDDDD 4	Удаление символов
1 1 1 abcaaaa abcdefg	MMRRRR 4	Замена символов
1 1 1 connect conehead	MMIRMRR 4	Полноценная работа
3 1 1 connect conehead	MMDMMDDIII 7	Разные цены операций. Удаление со вставкой выгоднее замены
1 1 1 abcdf abcde 2 1 e d	MMMMR 2	Вариант 2: замена на особый символ e, удаление f и вставка e, алгоритм выбирает замену
1 1 1 d c 0 5 c d	R 0	Вариант 2: замена на особый символ c бесплатна (цена 0), поэтому замена выгоднее удаления

```

● farid@Farid-M1-Mac src % python3 main.py
Enter replacement, insertion and deletion costs:
1 1 1

Enter first string (A):
abcde

Enter second string (B):
edcba

Add special replacer and special deletable character? (y/n)
y
Enter cost of replacing with special char, and cost of deleting special char:
1 2
Enter the special replacer char and the special deletable char:
a d

Word before operation abcde
Operation R on position 1
Replace a with e
Word after operation ebcde

Word before operation ebcde
Operation R on position 2
Replace b with d
Word after operation edcde

Word before operation edcde
Operation M - characters match, nothing to do

Word before operation edcde
Operation R on position 4
Replace d with b
Word after operation edcbe

Word before operation edcbe
Operation R on position 5
Replace e with a
Word after operation edcba

Result vs target:
edcba
edcba
Match!

      e  d  c  b  a
    0  1  2  3  4  5
a  1  1  2  3  4  4
b  2  2  2  3  3  4
c  3  3  3  2  3  4
d  5  4  3  4  3  4
e  6  5  4  4  4  4

Edit distance = 4
RRMRR
abcde
edcba
○ farid@Farid-M1-Mac src %

```

Рисунок 1 – Результат работы программы.

Исследование

Исследуем эффективность алгоритма Вагнера-Фишера для построения матрицы расстояний на различных объёмах входных данных.

Таблица 2. Исследование эффективности по времени.

Размер первого слова	Размер второго слова	Произведение размеров	Затраченное время (с)
10	10	100	0.00004
10	1.000	10.000	0.00322
1.000	10	10.000	0.00354
100	10.000	1.000.000	0.33267
10.000	100	1.000.000	0.34404
1.000	1.000	1.000.000	0.35040
1.000.000	10	10.000.000	4.02968

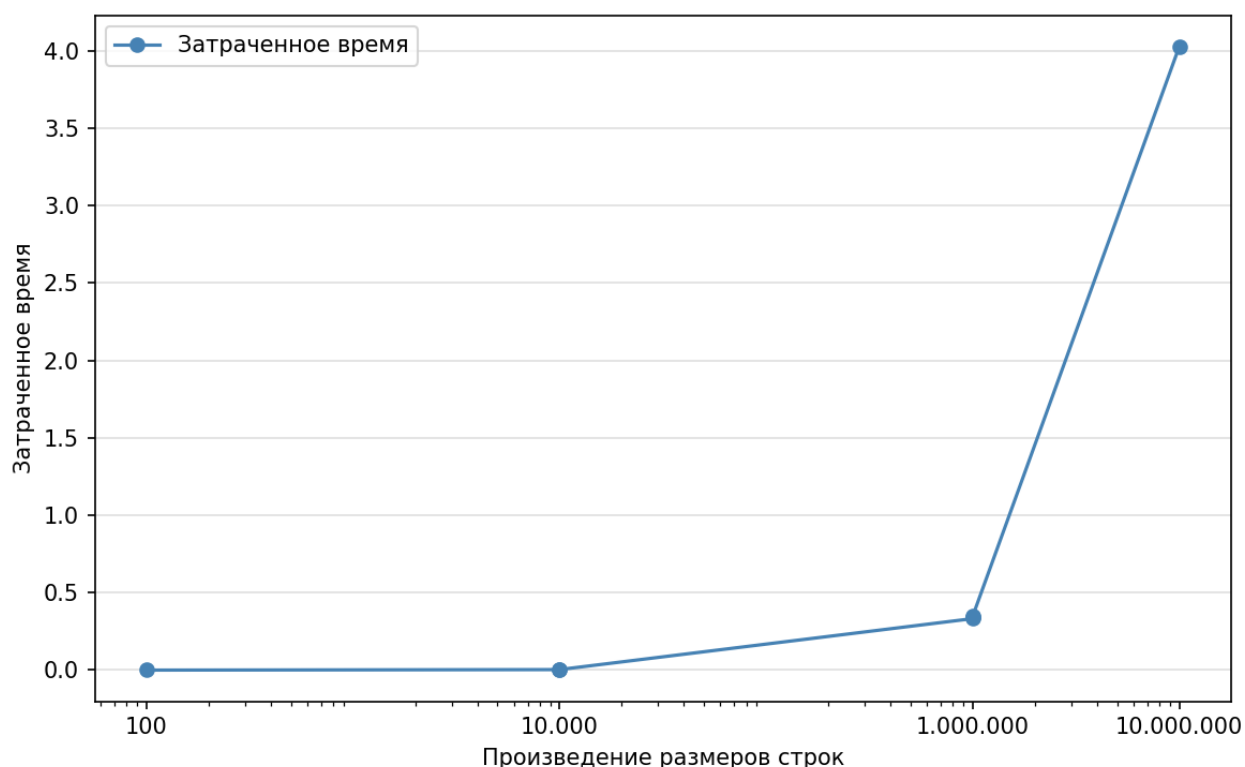


Рисунок 2 – График зависимости затраченного времени на получение матрицы расстояний от произведения размеров входных строк.

Можно сделать следующие выводы по исследованию:

1. Полученные результаты подтверждают временную оценку $O(n * m)$.
2. Результат по времени не зависит от того, какая из строк длиннее.

Выводы

В ходе выполнения лабораторной работы была изучена работа алгоритма Вагнера-Фишера. Решены задачи поиска матрицы расстояния Левенштейна и нахождения редакционного предписания.

ПРИЛОЖЕНИЕ

ИСХОДНЫЙ КОД ПРОГРАММЫ

Имя файла: main.py

```
import sys

def decide_costs(src_char, tgt_char, replace_cost, delete_cost,
                 r_char=None, r_price=None, d_char=None, d_price=None):
    r = r_price if (r_char and tgt_char == r_char) else replace_cost
    d = d_price if (d_char and src_char == d_char) else delete_cost
    return r, d

def get_distance_matrix(s1, s2, replace_cost, insert_cost, delete_cost,
                        r_char=None, r_price=None, d_char=None,
d_price=None):
    m, n = len(s1), len(s2)
    d = [[0] * (n + 1) for _ in range(m + 1)]

    for j in range(1, n + 1):
        d[0][j] = d[0][j - 1] + insert_cost

    for i in range(1, m + 1):
        _, del_cost = decide_costs(s1[i - 1], '', replace_cost, delete_cost,
                                   r_char, r_price, d_char, d_price)
        d[i][0] = d[i - 1][0] + del_cost

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                d[i][j] = d[i - 1][j - 1]
                continue
            r, del_cost = decide_costs(s1[i - 1], s2[j - 1], replace_cost,
delete_cost,
                                   r_char, r_price, d_char, d_price)

            d[i][j] = min(
                d[i - 1][j - 1] + r,
                d[i][j - 1] + insert_cost,
                d[i - 1][j] + del_cost
            )
    return d

def traceback_operations(d, s1, s2, replace_cost, insert_cost, delete_cost,
```

```

                                r_char=None,          r_price=None,          d_char=None,
d_price=None):
    i, j = len(s1), len(s2)
    solution = ''

    while i > 0 or j > 0:
        if i > 0 and j > 0 and s1[i-1] == s2[j-1] and d[i][j] == d[i-1][j-1]:
            solution += 'M'
            i -= 1; j -= 1
            continue

        r, del_cost = decide_costs(
            s1[i - 1] if i > 0 else '',
            s2[j - 1] if j > 0 else '',
            replace_cost, delete_cost,
            r_char, r_price, d_char, d_price
        )

        if i > 0 and j > 0 and d[i][j] == d[i-1][j-1] + r:
            solution += 'R'
            i -= 1; j -= 1
        elif j > 0 and d[i][j] == d[i][j-1] + insert_cost:
            solution += 'I'
            j -= 1
        else:
            solution += 'D'
            i -= 1

    return solution[::-1]

def check_solution(s1, s2, solution):
    s = list(s1)
    ptr = 0
    for op in solution:
        print(f"\nWord before operation {''.join(s)}")
        if op == 'M':
            print("Operation M - characters match, nothing to do")
            ptr += 1
            continue
        print(f"Operation {op} on position {ptr + 1}")
        if op == 'R':

```



```

        print(f"Replace {s[ptr]} with {s2[ptr]}")
        s[ptr] = s2[ptr]
    elif op == 'I':
        print(f"Insert {s2[ptr]}")
        s.insert(ptr, s2[ptr])
    elif op == 'D':
        print(f"Delete {s[ptr]}")
        del s[ptr]
        ptr -= 1
    ptr += 1
    print(f"Word after operation {''.join(s)}")

print("\nResult vs target:")
print(''.join(s))
print(s2)
if ''.join(s) == s2:
    print("Match!\n")
def print_matrix(d, s1, s2):
    print(' ', *[c for c in s2], sep=' ')
    for i, row in enumerate(d):
        label = ' ' if i == 0 else s1[i - 1]
        print(label, end=' ')
        for val in row:
            print(f"{val:<3}", end=' ')
        print()
    print()

def main():
    print("Enter replacement, insertion and deletion costs:")
    replace_cost, insert_cost, delete_cost = map(int, input().split())

    print("\nEnter first string (A):")
    s1 = input()

    print("\nEnter second string (B):")
    s2 = input()

    r_char = r_price = d_char = d_price = None

    print("\nAdd special replacer and special deletable character? (y/n)")
    if input() == 'y':

```

```

        print("Enter cost of replacing WITH special char, and cost of
deleting special char:")
        r_price, d_price = map(int, input().split())
        print("Enter the special replacer char and the special deletable
char:")
        r_char, d_char = input().split()

    d = get_distance_matrix(s1, s2, replace_cost, insert_cost, delete_cost,
                           r_char, r_price, d_char, d_price)

    solution = traceback_operations(d, s1, s2, replace_cost, insert_cost,
                                   delete_cost,
                                   r_char, r_price, d_char, d_price)

    check_solution(s1, s2, solution)
    print_matrix(d, s1, s2)

    print("Edit distance =", d[len(s1)][len(s2)])
    print(solution, s1, s2, sep='\n')

if __name__ == "__main__":
    main()

```